

AgentAttack: LLM agents for multi-strategy shilling attacks in recommenders

Chenyu Li ^a, Zongwei Wang ^a, Hao Ma ^a, Wei Li ^b, Huan Wu ^{c,d}, Yuqi Song ^e, Min Gao ^{a,*}

^a School of Big Data & Software Engineering, Chongqing University, 400044, Chongqing, China

^b Department of Plastic Surgery, Chongqing Emergency Medical Center, Chongqing University Central Hospital, 400000, Chongqing, China

^c Chongqing Municipal Facilities Safety Operation and Maintenance Application Technology Promotion Center, Chongqing City Vocational College, 402160, Chongqing, China

^d College of Environmental Science and Engineering, Tongji University, 200092, Shanghai, China

^e Department of Computer Science and Engineering, University of South Carolina, 29201, Columbia, SC, USA

ARTICLE INFO

Keywords:

Recommender systems
Shilling attacks
LLM agents
Security

ABSTRACT

Recommender systems are crucial for filtering information in e-commerce, social media, and content platforms, but they are increasingly vulnerable to shilling attacks. Traditional fixed-pattern and hybrid attack methods are inflexible and fail to adapt to different recommendation systems, while end-to-end learning approaches are often overfitted to specific models, limiting their generalization. Recent advancements in large language models (LLMs) have enabled intelligent agents to autonomously plan and execute sophisticated, adaptive attacks, though their application in real-world recommendation threats remains underexplored.

To fill the gap and rigorously assess the resilience of modern recommender systems, we propose AgentAttack, a novel framework that integrates LLM agents to generate and execute multi-strategy shilling attacks. Unlike static or narrowly trained attack mechanisms, AgentAttack distinguishes itself by its ability to select and compose attack strategies based on real-time reasoning and historical feedback, rather than relying on predefined, fixed strategies. The framework comprises four synergistic modules that generate fake users, plan attack strategies, incorporate feedback, and implement attacks, together enhancing the flexibility and effectiveness of attack coordination. Experimental results show that AgentAttack outperforms traditional and end-to-end learning-based attack methods, achieving higher attack hit rates against different recommender systems on multiple real-world datasets, including Douban.

1. Introduction

Recommender systems, as the core engines of information filtering, have become deeply integrated into various domains, including e-commerce, social networks, and content platforms (Aggarwal et al., 2016; Lu et al., 2015). Leveraging advancements in big data technologies, machine learning, and distributed computing, modern recommender systems (Deldjoo et al., 2023; Wang et al., 2023) not only address the challenge of information overload but also transition from a one-size-fits-all approach to personalized experiences through techniques such as collaborative filtering (Schafer et al., 2007), deep learning (Zhang et al., 2019), and real-time feedback mechanisms (Roy & Dutta, 2022). These systems now play a central role in influencing user decision-making, as they aggregate and process diverse data to provide personalized recommendations (Jesse & Jannach, 2021; Zaharaddeen & Mukhtar, 2024).

However, their pervasive adoption has made them prime targets for malicious manipulation. Among these, shilling attacks (Gunes et al., 2014; Si & Li, 2020; Wang et al., 2024e) are especially prominent, involving the injection of fake users and biased feedback to distort recommendation outputs. While traditional studies have explored the impact of isolated attack types (Hurley et al., 2007; Lam & Riedl, 2004; O'Mahony et al., 2005), these fixed-pattern approaches exhibit diminished effectiveness against complex modern recommenders. Hybrid attacks (Zhang, 2011), which combine multiple strategies (Fig. 1a), still rely heavily on manually designed patterns and thus lack flexibility. End-to-end neural approaches (Lin et al., 2020, 2022; Wang et al., 2024c,d; Wu et al., 2021a) automate attack profile generation (Fig. 1b), but suffer from overfitting, poor transferability, and the need for extensive training. A recent survey has revealed that existing attack methods struggle to adapt to the complexity of modern recommendation systems, especially those with multi-domain and decentralized archi-

* Corresponding author.

E-mail addresses: chenyu_li@stu.cqu.edu.cn (C. Li), zongwei@cqu.edu.cn (Z. Wang), ma_hao@cqu.edu.cn (H. Ma), liwei00128@cqu.edu.cn (W. Li), wuhuan@cqcvc.edu.cn (H. Wu), yuqis@email.sc.edu (Y. Song), gaomin@cqu.edu.cn (M. Gao).

<https://doi.org/10.1016/j.eswa.2026.131163>

Received 19 June 2025; Received in revised form 14 November 2025; Accepted 6 January 2026

Available online 12 January 2026

0957-4174/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

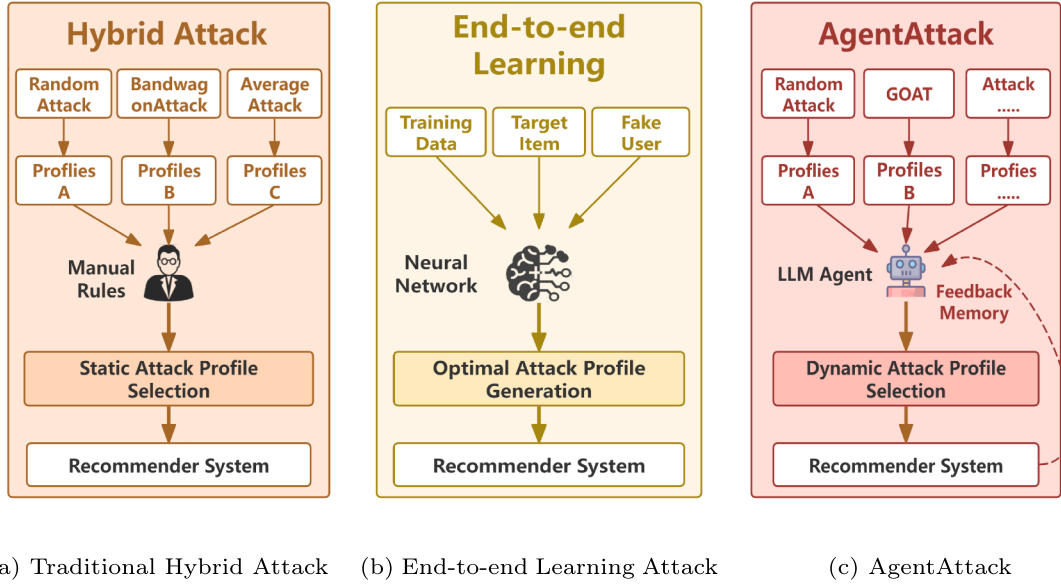


Fig. 1. Comparison of attack methods.

tures, highlighting the limitations of current strategies in evolving environments (Wang et al., 2024e). The experiments have shown that traditional attack methods, such as Probe Attack and Bandwagon Attack, exhibit significant performance variations across different recommendation models, and their effectiveness noticeably declines when scaling to larger datasets or more complex architectures.

Recent advancements in large language models (LLMs) (Naveed et al., 2023) have unlocked new possibilities for strategy generation and contextual reasoning. While LLMs have propelled the evolution of recommendation systems themselves, they also offer powerful tools for adversarial attacks. LLM-based agents (Cheng et al., 2024; Wang et al., 2024b; Xi et al., 2025), equipped with advanced decision-making, planning, and learning capabilities, can interact with external systems through APIs and refined prompts to generate, evaluate, and optimize attack strategies. Despite their capabilities, the use of LLM agents for real-world and multi-round hybrid shilling attacks remains underexplored (Ge et al., 2024).

To situate our work formally, we consider a recommender system parameterized by Θ that learns from user-item interactions. The problem we address is the generation of dynamic shilling attacks: an LLM-powered adversary aims to inject a set of fake profiles U' with crafted interactions D' to perturb Θ , thereby manipulating recommendations. This process is formulated as a multi-round interaction where the agent, based on the system state s_t , plans and executes an attack action a_t , and then refines its approach using received feedback r_t . We hypothesize that by leveraging the contextual reasoning and planning inherent in LLM agents, the resulting strategies will surpass the effectiveness, transferability, and scalability of existing baselines in complex environments.

To address this methodological gap and evaluate the proposed hypothesis, we propose AgentAttack, a novel hybrid shilling attack framework powered by LLM agents, illustrated in Fig. 1c. Unlike conventional approaches, AgentAttack uses LLM agents as intelligent routers to dynamically compose and execute attack strategies based on real-time reasoning and historical outcomes. Similar to the use of AI models in privacy-preserving transfer learning for community detection, where strategies are applied across decentralized networks (Rosli, 2025), our work leverages LLM agents to generate flexible attack strategies capable of contending with the evolving complexities of modern recommender systems. The framework integrates memory modules and reflective feedback loops, enhancing its adaptability to a range of recommender ar-

chitectures and defense strategies. As highlighted in recent studies, ensuring precision and clarity in the design of AI-driven frameworks is crucial for their effective integration into complex cybersecurity environments (Ali & Ghanem, 2025).

Specifically, AgentAttack selects multiple candidate attack methods and identifies the user groups most susceptible to each. The agent then constructs combinations of fake users through routing decisions that maximize attack effectiveness. The framework integrates four key components: an attack profile module for generating diverse fake users, a planning module that leverages LLM reasoning to optimize strategic decisions, a feedback memory module that refines attack selection through iterative learning, and an action module for implementing attacks in the target system. By selecting multiple candidate attack methods and identifying the most vulnerable user groups, AgentAttack autonomously constructs effective attack combinations. Furthermore, we investigate how different prompt engineering techniques influence the controllability and efficiency of agent-driven attacks.

It is important to emphasize that the goal of this research is not to facilitate adversarial misuse, but to “promote defense through offense”. By systematically deconstructing how intelligent agents can orchestrate complex shilling attacks, this work provides actionable insights for designing the next generation of recommendation systems with intrinsic robustness and security.

The main contributions and innovations of this paper are as follows:

- To the best of our knowledge, we propose an innovative LLM agent-driven shilling attack framework, overcoming the rigidity, inflexibility, and limited generalization of prior fixed-rule, hybrid, and end-to-end methods via multi-strategy planning.
- We design a prompt-driven fake user generation mechanism, using 10 distinct prompts to guide diverse yet goal-consistent attack behaviors, enabling rich comparative evaluation.
- Extensive experiments across multiple datasets show that our adaptive strategy outperforms traditional fixed-pattern and end-to-end learning-based baselines, advancing both attack realism and LLM applications in adversarial recommendation research.
- The insights gained from this work could contribute to the development of more resilient recommender systems, helping to safeguard user trust and improve the ethical use of recommendation algorithms in diverse industries.

The structure of this paper is as follows: [Section 2](#) analyzes related research. [Section 3](#) establishes preliminary foundations, including recommender systems, shilling attack methods, and LLM-based agents. [Section 4](#) details the AgentAttack methodology. [Section 5](#) presents comprehensive experimental evaluation and comparative analysis across multiple datasets. [Section 6](#) summarizes key contributions and future directions.

2. Related works

2.1. Shilling attacks in recommender systems

Recommender systems function as personalized information filters, analyzing users' historical interactions to predict items of potential interest. Over the decades, research in this field has expanded across numerous application domains and embraced diverse algorithmic approaches ([Park et al., 2012](#)). These systems have evolved from traditional algorithms to sophisticated approaches leveraging Graph Neural Networks (GNNs), which naturally model the graph-structured data inherent in user-item interactions ([Gao et al., 2022](#)). Notable examples include Neural Graph Collaborative Filtering (NGCF) ([Wang et al., 2019](#)) and LightGCN ([He et al., 2020](#)). While modern recommendation algorithms focus on accuracy, diversity, fairness, and user privacy through techniques like self-supervised learning ([Wu et al., 2021b](#)) and federated recommendation mechanisms ([He et al., 2024](#)), they face significant security challenges.

Among these challenges, shilling attacks represent a critical threat where malicious users manipulate recommendation results by injecting fake data or interfering with model training ([Si & Li, 2020](#)). As [Wang et al. \(2024e\)](#) explain in their comprehensive survey, these attacks can be systematically understood through a four-stage pipeline: setting attack goals, assessing attacker capabilities, analyzing victim architecture, and implementing poisoning strategies. In these attacks, attackers typically create attack profiles consisting of feature items, randomly filled items, target items, and unrated items ([Sundar et al., 2020](#)). Mainstream basic attack types include: Random Attack, a simple method that rates randomly filled items using the system's average rating ([Lam & Riedl, 2004](#)); Average Attack, which improves attack effectiveness by using values close to each item's average rating ([Burke et al., 2006](#)); Bandwagon Attack, which enhances similarity by leveraging the popularity of trending items ([O'Mahony et al., 2005](#)); and Probe Attack, which builds profiles highly similar to real users by analyzing system responses ([Hurley et al., 2007](#)).

With the advancement of research, end-to-end learning attack methods have emerged as sophisticated approaches that automatically optimize attack strategies through neural network and agent-based model training. GOAT ([Wu et al., 2021a](#)) targets Graph Neural Network recommender systems by designing malicious user interactions guided by gradients; AUSH ([Lin et al., 2020](#)) uses generative adversarial networks to create fake user profiles; the Contrastive Learning Recommendation Attack (CLear) ([Wang et al., 2024d](#)) reveals that recommender systems based on contrastive learning exhibit particular vulnerabilities to poisoning attacks aimed at promoting target items; InfoAtk ([Ma et al., 2024](#)) balances attack stealthiness and effectiveness by using contrastive learning to maintain representation consistency while enhancing target item rankings; the Uplift-guided Budget Allocation (UBA) ([Wang et al., 2024c](#)) method optimizes fake user budget allocation based on heterogeneous treatment effects to enhance attack effectiveness in recommender systems. These methods perform excellently in specific scenarios, but they are often designed for specific algorithms and lack universality.

To address the limitations of single attack methods, researchers have begun exploring hybrid attack strategies. The Hybrid Attack ([Zhang, 2011](#)) cleverly combines the characteristics and advantages of Average Attack and Bandwagon Attack. Subsequent research ([Yu et al., 2017](#)) further proposed a novel hybrid attack method that integrates both rating attack and relationship attack features, which is more powerful than

traditional rating attacks in model-based social recommendation systems and can be effective without relying on feedback links from normal users, achieving effective attacks through lower-cost unidirectional links alone. Despite these significant advances in composite strategies, they still lack intelligent decision support systems to automatically select and combine attack methods most suitable for the current scenario, and have not fully considered the optimization of attack resource allocation under resource-constrained situations.

To summarize and better compare the advantages and limitations of various attack methods discussed above, we provide the following table. [Table 1](#) highlights the key characteristics of three major attack types: traditional fixed-pattern attacks, hybrid attacks, and end-to-end learning attacks.

Taken together, existing attack methods have yielded effective results in specific scenarios, but they often lack intelligent decision support systems and cannot automatically select and adjust attack strategies based on the characteristics of different recommender systems, resulting in limited generalization and scalability. Clearly identifying system vulnerabilities and shortcomings is crucial to improving system resilience. By addressing these weaknesses, it is possible to design more flexible and robust defense mechanisms, thus advancing security-oriented research ([Hussain, 2024](#)). To this end, we propose the AgentAttack framework, aiming to fill the gap in attack research and promote the development of recommender system defenses. The framework can intelligently select and adjust attack strategies based on real-time feedback, thereby enhancing the flexibility of attacks and improving their ability to adapt to various environments.

2.2. The evolution and application of LLM agents

Large Language Models (LLMs) have brought new opportunities and possibilities to the field of strategy generation. Since the transformer architecture ([Vaswani et al., 2017](#)) was proposed, models ranging from T5 ([Raffel et al., 2020](#)) and GPT-3 ([Brown et al., 2020](#)) to the latest LLaMA ([Touvron et al., 2023](#)), PaLM ([Chowdhery et al., 2023](#)), and GPT-4 have demonstrated increasingly powerful language understanding, reasoning, and zero-shot learning capabilities.

The rapid development of LLMs has propelled groundbreaking advances in LLM agent research. As next-generation intelligent systems, LLM-agents utilize language models as their core "brain" while integrating memory, planning, and tool-calling capabilities, demonstrating remarkable potential for completing complex tasks ([Wang et al., 2024b](#); [Xi et al., 2025](#)). The ReAct paradigm significantly enhances decision-making through its "observe-think-act" cycle mechanism ([Yao et al., 2023](#)), while the Reflexion framework innovatively introduces a reflection mechanism, enabling agents to develop self-improvement capabilities ([Shinn et al., 2023](#)). In recent research, ExpeL agents have achieved a breakthrough learning paradigm—extracting knowledge from experience and transferring it to new task scenarios without requiring parameter updates ([Zhao et al., 2024](#)). These advances demonstrate the enormous potential of LLM-agents in planning, reasoning, and learning.

Research integrating LLMs has shown explosive growth, and the combination with recommender systems represents a highly promising research direction. InteRecAgent ([Huang et al., 2023](#)) uses LLM-based agents as a "brain" and recommendation models as "tools" to process user needs through querying, retrieval, and ranking. It features "shared candidate bus" and "user profile" modules to enhance memory, excelling in knowledge-limited scenarios. Agent4Rec ([Zhang et al., 2024](#)) uses LLMs to create agents simulating users in recommendation contexts. These agents, with profiles, memory, and action modules, display diverse social traits and preferences, evaluating recommendation performance, simulating filter bubbles, and revealing causal relationships while reducing dependence on A/B testing.

LLM-based agents may also pose security risks. Research ([Wang et al., 2025a](#)) demonstrates that LLM agents can be strategically deployed to attack ID-free recommenders by synthesizing deceptive text

Table 1
Comparison of attack types.

Attack Type	Advantages	Disadvantages	Applicable Scenarios
Fixed-Pattern Attacks	Simple to implement; low resource consumption	Lack of adaptability; limited effectiveness; easy to detect	Simple recommendation systems; low complexity attack scenarios
Hybrid Attacks	Combines multiple strategies, increases attack diversity	Lack of automated decision support; depends on manual design; complex implementation	Slightly more complex; recommendation systems that require flexible strategies
End-to-End Learning Attacks	Can automatically generate complex attack strategies	Large training data requirements; prone to overfitting; limited generalization ability	Complex, model-specific recommendation systems; deep learning scenarios

descriptions that mimic popular items. This highlights the dual nature of LLM capabilities—they can both enhance recommender systems and be exploited to attack them. LLM agents in recommendation systems still face challenges in terms of explainability and trustworthiness. Understanding both the beneficial uses and potential misuses of LLM agents is crucial for developing more robust recommender systems.

In current research, LLMs have been widely applied in various fields such as task planning and decision support. However, the application of LLMs to shilling attacks in recommender systems is still in its early stages, especially when it comes to optimizing attack strategies for complex recommendation environments. The AgentAttack framework proposed in this paper introduces LLM-based agents to implement more sophisticated and efficient shilling attacks. The following sections will provide a detailed description of the design and implementation of AgentAttack.

3. Preliminary

3.1. Recommender systems based on implicit feedback

Modern recommender systems are widely applied in information filtering, helping users discover interesting content in information-overloaded environments (Aggarwal et al., 2016; Lu et al., 2015). For implicit feedback scenarios, we define the user-item interaction dataset as $\mathcal{D} = \{(u, i) \mid u \in \mathcal{U}, i \in \mathcal{I}_u\}$, where $\mathcal{U}$ denotes the user set, $\mathcal{I}$ denotes the item set, and $\mathcal{I}_u$ denotes the subset of items that user u has interacted with. The interaction data can be represented as a sparse matrix $\mathbf{R} \in \mathbb{R}^{|\mathcal{U}| \times |\mathcal{I}|}$, where element $r_{u,i} = 1$ indicates an interaction exists, otherwise, 0.

Recommender systems for implicit feedback typically employ various approaches to model user preferences. Memory-based collaborative filtering directly utilizes historical interaction data to make recommendations. Model-based approaches such as Matrix Factorization (MF) (Hu et al., 2008) learn latent representations by factorizing the user-item interaction matrix:

$$\mathbf{R} \approx \mathbf{P}\mathbf{Q}^T, \quad (1)$$

where $\mathbf{P} \in \mathbb{R}^{|\mathcal{U}| \times d}$ is the user embedding matrix and $\mathbf{Q} \in \mathbb{R}^{|\mathcal{I}| \times d}$ is the item embedding matrix. The predicted preference score is calculated as:

$$\hat{r}_{u,i} = \mathbf{p}_u^T \mathbf{q}_i. \quad (2)$$

For implicit feedback data, Bayesian Personalized Ranking (BPR) (Pan & Chen, 2013) provides an effective optimization framework:

$$\mathcal{L}_{BPR} = \sum_{(u,i,j) \in \mathcal{O}} -\ln \sigma(\hat{r}_{u,i} - \hat{r}_{u,j}) + \lambda \|\Theta\|_2^2, \quad (3)$$

where $\mathcal{O} = \{(u, i, j) \mid (u, i) \in \mathcal{D}, (u, j) \notin \mathcal{D}\}$ is the training triplet set.

Neural network-based recommender systems (He et al., 2017) have also demonstrated strong performance by replacing the linear inner product with neural architectures, enabling more complex user-item relationship modeling.

3.2. Shilling attacks against recommender systems

Attack Objectives and Types. Shilling attacks aim to manipulate recommender system results by injecting fake user profiles, typically targeting an increase in specific item exposure (Gunes et al., 2014; Si & Li, 2020). Based on attack objectives, they can be categorized into:

- *Push attack*: Increases the recommendation frequency of target items, making them more discoverable by users.
- *Nuke attack*: Decreases the recommendation frequency of competing items, reducing their exposure.
- *Targeted attack*: Manipulates recommendation results only for specific user groups.

Attack Knowledge and Capabilities. According to the attacker's understanding of the recommender system, attacks can be classified as (Wang et al., 2024e):

- *White-box attack*: The attacker has access to all components of the recommender system, including model architecture, parameters, and training data.
- *Gray-box attack*: The attacker knows the type of recommendation algorithm but cannot access the complete model parameters or training data.
- *Black-box attack*: The attacker can only observe system inputs and outputs without knowledge of internal mechanisms, representing the most common attack type in real-world scenarios.

Attack Profile Structure. A typical attack profile in a shilling attack consists of the following components (Gunes et al., 2014):

- *Selected Items (SI)*: Items specifically chosen by the attacker to rate according to the attack strategy.
- *Filler Items (FI)*: Randomly selected items that are rated to make the profile appear realistic and avoid detection.
- *Target Items (TI)*: Items that the attacker aims to promote (for push attacks) or demote (for nuke attacks).
- *Unrated Items (UI)*: Items that receive no rating from the fake user.

The attack implementation typically involves constructing a fake user interaction matrix $\mathbf{F} \in \mathbb{R}^{n \times |\mathcal{I}|}$ (where n is the number of fake users and $\mathcal{I}$ denotes the item set), which is then injected into the original interaction matrix to form the poisoned dataset:

$$\mathbf{R}_{\text{poisoned}} = \begin{bmatrix} \mathbf{R} \\ \mathbf{F} \end{bmatrix}. \quad (4)$$

3.3. LLM-based agents

LLM-based agents utilize large language models as the core cognitive unit while integrating multiple functional modules to achieve complex task processing (Cheng et al., 2024; Wang et al., 2024b; Xi et al., 2025). A typical LLM agent architecture consists of four key components (Wang et al., 2024b; Xi et al., 2025):

- *Profile Module*: Establishes the agent's foundational attributes, including its identity positioning, behavioral standards, and areas of expertise, enabling it to maintain consistent performance.

- **Memory Module:** Responsible for information storage and retrieval, which helps the agent form continuous cognition and refer to past experiences to guide decisions in new contexts, resulting in more personalized responses.
- **Planning Module:** Functions as the agent's thinking center, handling problem analysis and solution design. Through reasoning, this module determines the most appropriate execution strategy.
- **Action Module:** Transforms abstract plans into concrete behaviors, executes instructions, and collects environmental feedback to evaluate action effectiveness.

These modules operate collaboratively and continuously optimize agent behavior through a reflection mechanism (Reflexion) (Shinn et al., 2023). Unlike traditional fixed-rule attack methods, LLM agents can dynamically select and combine attack strategies, enabling more flexible real-time decision-making. This attack decision process is continuously adjusted through iterative feedback and reasoning mechanisms, rather than relying on pre-trained model parameters. In contrast, end-to-end neural network methods learn attack patterns through training, and the model parameters θ obtained during training are used for subsequent decisions:

$$R_{\text{fake}} = f_{\theta}(X_{\text{user}}, X_{\text{item}}, Z_{\text{attack}}), \quad (5)$$

where X_{user} represents the user's features, X_{item} represents the item's features, Z_{attack} represents the attack strategy or attack target information, θ denotes the model parameters obtained through training, and R_{fake} represents the generated fake user rating data.

Although the model can be fine-tuned for adjustments, end-to-end methods typically rely on fixed parameters learned during the training phase, and adjustments during the inference phase are relatively limited. The adjustment process still requires training, lacking continuous and immediate adaptability. In contrast, in our approach, the LLM agent selects and combines attack strategies through the following formal representation, demonstrating its flexible real-time adjustment capability:

$$\mathcal{U}_{\text{fake}} = \text{Agent}(\mathcal{A}, \mathcal{M}, f), \quad (6)$$

where $\mathcal{U}_{\text{fake}}$ represents the selected fake user profile set, $\mathcal{A}$ represents the attack library containing various attack methods, $\mathcal{M}$ is the agent's memory module, and f represents the target recommendation system model. Through this process, the LLM agent can adjust its strategy in real-time to respond to different recommendation system environments and feedback, enabling more efficient attacks.

The composite poisoned dataset is then constructed as:

$$\mathcal{D}_{\text{poisoned}} = \mathcal{D} \cup \mathcal{D}_{\text{fake}}, \quad (7)$$

where $\mathcal{D}_{\text{fake}} = \{(u, i) \mid u \in \mathcal{U}_{\text{fake}}, i \in \mathcal{I}\}$.

This approach enables AgentAttack to adjust user selection strategies based on recommender system feedback, breaking through the limitations of traditional attack methods.

4. Methods

In this section, we introduce AgentAttack, an innovative framework that leverages LLM-based agents to generate hybrid shilling attack strategies against recommender systems. We first present the overall framework architecture, followed by a detailed explanation of the design and functionality of each core component.

4.1. Framework overview

Previous methods have limited effectiveness when facing complex defense strategies, due to their lack of adaptive capabilities and intelligent decision-making mechanisms. To overcome this limitation, there is a need for a framework that can combine the advantages of multiple attack strategies and adjust attack methods. Research (Cheng et al., 2024; Wang et al., 2024b; Xi et al., 2025) indicates that LLM-based agent architectures typically consist of four core modules: Profile Module, Memory

Module, Planning Module, and Action Module. This architecture has been proven to have significant advantages across multiple domains, enabling agents to simulate human thinking processes, accumulate experience, and adapt to complex environments.

After deeply analyzing the limitations of attack methods and the advantages of LLM agent architectures, we present a key insight: the four-module architecture of LLM agents can be redesigned for shilling attack scenarios, achieving unprecedented attack capabilities. Based on this insight, we designed AgentAttack, maintaining the same four-module structure, but with each module specifically designed for attack scenarios.

Attack Profile Module. This module is responsible for building an attack library and generating fake users. It addresses the problem that individual attack methods struggle against complex defense mechanisms, providing the system with diverse attack options.

Planning Module. The Planning Module represents the cognitive core of AgentAttack, functioning as an intelligent routing engine that transforms attack strategy selection into reasoning-driven decisions. This module addresses a critical gap in existing attack methodologies: the lack of intelligence in strategy selection.

Feedback Memory Module. This module records and analyzes historical attack data, accumulating experience to identify successful patterns and avoid repeating failed strategies, solving the problem that common hybrid attack methods cannot learn from past experiences.

Action Module. This module transforms strategies into actual actions, integrating fake users into the target system. Through a data fusion process, it combines the original matrix with the fake user matrix into a poisoned dataset, ensuring fake users blend seamlessly without detection.

System Integration. These four modules form a closed-loop system, as indicated by the arrows in Fig. 2: the Attack Profile Module provides attack data to the Planning Module; the Planning Module makes decisions based on historical results and passes selected users to the Action Module; the Action Module executes attacks and generates performance data, stored by the Feedback Memory Module and provided to the Planning Module for future decisions. This design achieves a complete cycle from data to decision, from decision to action, and from action to learning, enabling AgentAttack to continuously improve itself.

4.2. Attack profile module

4.2.1. Attack library construction

In traditional agent architectures, the description module defines the agent's role characteristics to influence its behavior (Wang et al., 2024b). For recommendation system attack tasks, we recognize that the key challenge is: different attack methods each have their advantages, but a single method struggles to handle all defense scenarios. We innovatively extended the concept of "profile" to the characteristic description of attack methods, constructing a comprehensive attack method library.

This design decision stems from in-depth analysis of existing attack methods. We can find that each method has its unique advantages and limitations, which prompted us to design a framework capable of combining the advantages of multiple attack methods. By integrating diverse attack methods including Random Attack (Lam & Riedl, 2004), Average Attack (Burke et al., 2006), Bandwagon Attack (O'Mahony et al., 2005), Probe Attack (Hurley et al., 2007), and advanced methods such as GOAT (Wu et al., 2021a) and AUSH (Lin et al., 2020), we built a comprehensive attack library.

Building this diversified attack library is a complex task, requiring not only the implementation of various attack algorithms but also the assurance that they can work collaboratively within a unified framework. To address this technical challenge, we implemented a dynamic loading mechanism, enabling the framework to flexibly integrate different attack methods and support future extensions.

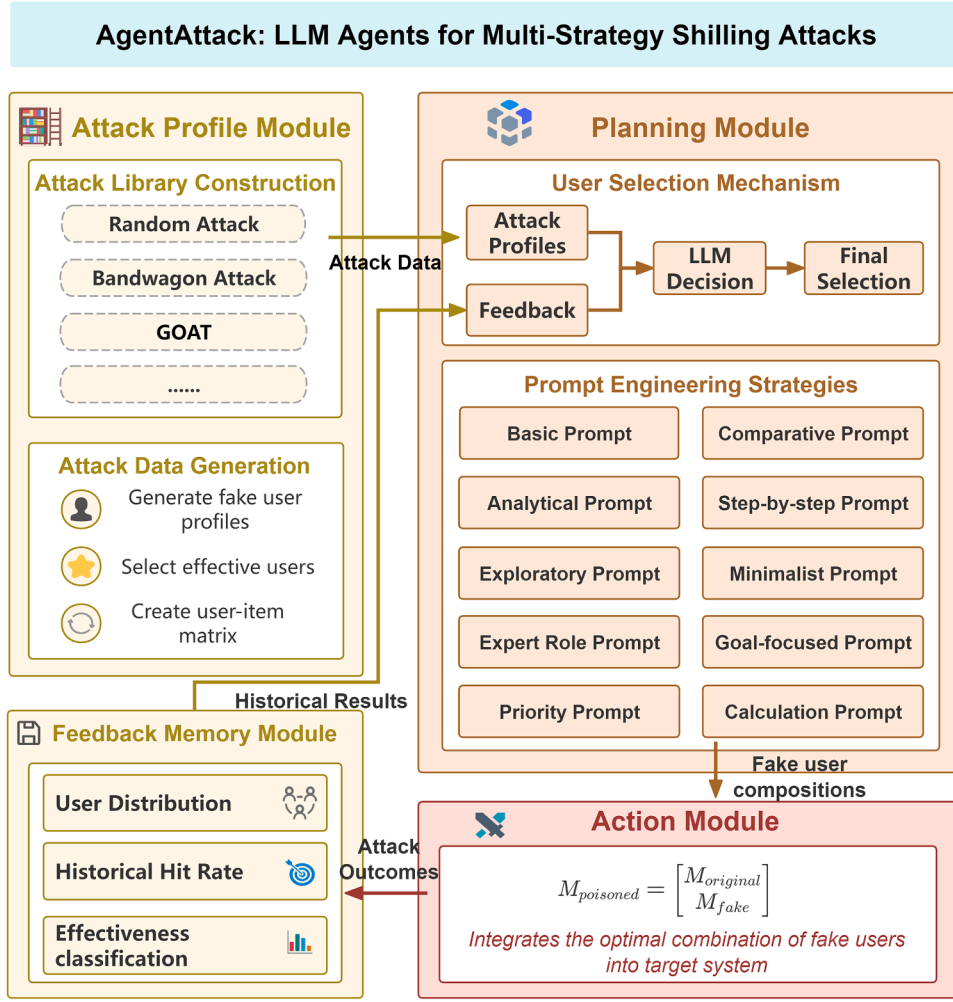


Fig. 2. The framework of AgentAttack.

4.2.2. Attack data generation

After building the attack library, we need to generate high-quality fake user profiles for each attack method. Simply generating all possible fake users would result in many inefficient or easily detectable profiles, reducing the overall attack effectiveness. To address this challenge, we propose a quality-first user selection strategy.

To simulate real user behavior and adapt to different attack scenarios, the system first loads modules to generate a large number of candidate fake users as needed for attacks in the library. Each fake user profile contains synthetic interaction patterns between target items and popular items, while introducing random noise to enhance behavioral diversity. To ensure that these fake users contribute maximally to the attack's effectiveness, we use the hit rate as the primary evaluation metric to quantify the effectiveness of the fake user configurations.

To prevent inefficient or easily detectable fake users from consuming resources, AgentAttack employs an iterative evaluation mechanism to filter the optimal subset. Specifically, the system randomly samples 2/3 of the users from the candidate pool, generates a poisoned dataset, trains a recommendation model, and quantifies the attack effectiveness through hit rate. If the current combination fails to improve effectiveness, an elimination mechanism is triggered; if the effect is significant, the elimination counter is reset and the best combination is updated. After multiple iterations, the user set with the highest hit rate is retained, ensuring efficient use of attack resources.

Directly modifying original data can trigger integrity checks, while simply appending fake users may lead to abnormal ID distributions.

AgentAttack overlays fake user interaction data onto the original dataset through matrix operations and reconstructs the training files while preserving the integrity of validation and test sets, building structured interaction data.

The attack data generation process involves extensive simulation and effectiveness evaluation work, ensuring that only high-quality fake users enter the subsequent selection pool. The generated fake user profiles contain complex synthetic interaction patterns that are carefully designed to both effectively promote target items and avoid detection by defense systems.

4.3. Planning module

4.3.1. User selection mechanism

In order to give the attack method intelligent decision-making capabilities that can select the optimal attack strategy based on specific situations, we designed a planning module based on LLM agents, enabling the system to select the optimal attack combination according to historical data and current context.

The core challenge of the user selection mechanism is organizing diverse heterogeneous attack data in an LLM-understandable format while guiding LLMs to make effective decisions. To overcome this challenge, we designed three key steps: data processing, LLM decision formulation, and final selection. Together, these steps form a comprehensive intelligent attack planning framework.

In the data processing phase, the attack data from the attack profile module and historical results from the feedback memory module are integrated into a structured format, enabling LLMs to process them effectively. This integration is crucial because it converts heterogeneous attack data into a unified representation, provides contextual information on past performance as a basis for future decisions, and creates standardized input formats to maximize LLMs' reasoning capabilities.

In the LLM decision-making phase, the system does not use simple heuristic methods, but instead leverages the advanced reasoning capabilities of large language models to analyze data and generate strategic decisions. Through multiple rounds of experiments, we designed an optimized system prompt that precisely guides the reasoning process of the LLM agent:

*You will receive information about fake users constructed by various attack types and their **hit rates**. Please help me select a total of $< n >$ constructed fake users with the goal of increasing the overall hit rate. Please note:*

- 1) The goal is to **maximize the hit rate**. The total number of all selected users must equal exactly $< n >$;
- 2) Historical selection results and feedback: $< M >$. You can refer to **Hit Rate Category** from the history.
- 3) Historical selections are merely for reference to maximize hit rate;
- 4) Results must strictly follow the format: $< Format >$

This prompt design balances guidance and flexibility, providing LLMs with clear task objectives and constraints while allowing them to exercise their creative problem-solving abilities.

The final selection phase involves validating and implementing LLMs' recommendations, ensuring they meet all system requirements. This includes verifying that the selected number of users precisely meets the requirements and strictly validating LLMs' output through established functions to ensure it follows the required format and constraints before proceeding with attack implementation. The [Algorithm 1](#) demonstrates the complete process of the planning module.

Algorithm 1: LLM Attack Planning Module.

Input: Attack configuration file P , historical feedback M , target number of users n

Output: User selection dictionary S

Function LLMAttackPlanning(P, M, n):

Step 1: Data Preparation ;

foreach attack type i in attack type list **do**

$P[i] \leftarrow (\text{userlist}(i), \text{hitrate}(i))$;

$M \leftarrow \text{get historical selection records}()$;

Step 2: LLM Decision Making ;

 prompt $\leftarrow \text{construct prompt}(P, M, n)$;

 response $\leftarrow \text{LLM}(\text{prompt})$;

$S \leftarrow \text{parse response}(\text{response})$;

Step 3: Output Validation ;

while total number of users(S) $\neq n$ **or**

$\neg \text{validate user existence}(S, P)$ **do**

 response $\leftarrow \text{LLM}(\text{adjust prompt}(\text{prompt}, \text{error message}))$

 ;

$S \leftarrow \text{parse response}(\text{response})$;

return S ;

4.3.2. Prompt engineering strategies and techniques

To further optimize LLMs' decision-making capabilities, we designed and evaluated ten different prompt variants. Below are their design characteristics and rationales:

Basic Prompt. Provides direct instructions focusing on maximizing hit rate. Serves as a baseline reference, removing complex logic to test LLMs' basic decision-making abilities under simple instructions and to observe whether they can efficiently complete tasks without auxiliary information.

*You will receive information about fake users constructed by various attack types along with their **hit rates**. Please help select a total of $< n >$ fake users with the goal of **maximizing the overall hit rate**.*

Comparative Prompt. Encourages comparison of attack types before making selections. By enforcing comparative analysis, this guides LLMs to comprehensively evaluate the strengths and weaknesses of various attack types, avoiding local optimal solutions that might overlook globally optimal combinations, thus enhancing the comprehensiveness of decision-making.

*Please compare the hit rates of the following attack types and select a total of $< n >$ users to achieve the **highest total hit rate**.*

Analytical Prompt. Emphasizes data-driven decision-making, promoting in-depth analysis of historical data. This stimulates LLMs' critical analysis capabilities regarding historical feedback, encouraging dynamic strategy adjustments based on real-time data rather than relying on past experience, thus enhancing data-driven adaptability.

*You will receive data about fake users from various attack types. Based on data analysis, please select a total of $< n >$ fake users to **maximize the overall hit rate**.*

Step-by-step Prompt. Guides the agent through a structured decision-making process, providing structured steps. By constraining LLMs' thinking path through a phased process, it reduces randomness in decision-making, ensures logical coherence, and is particularly suitable for step-by-step reasoning in complex scenarios.

*You will receive information about fake users from various attack types. Please follow these steps to select a total of $< n >$ fake users to **maximize the overall hit rate**: 1) Calculate the average hit rate for each attack type. 2) Begin selecting users from the attack types with the **highest hit rates**.*

Exploratory Prompt. Encourages trying different user combinations, enhancing innovative decision-making. This guides LLMs to actively explore potentially efficient combinations in the solution space, avoiding getting trapped in local optima, and stimulating innovative decision-making.

*You need to select a total of $< n >$ fake users from various attack types. Try exploring different combinations of users to **maximize the hit rate**.*

Minimalist Prompt. Provides only necessary instructions, removing redundant descriptions. This reduces information noise interference for LLMs, testing their generalization ability and focus on core objectives with minimized input.

Select a total of $< n >$ fake users from various attack types to **maximize the overall hit rate**.

Expert Role Prompt. Positions the agent as a data analyst, emphasizing professional identity. Through role-playing, this activates LLMs' internalized domain knowledge repository (such as data analysis methodologies), enhancing the professionalism and credibility of decisions, and mimicking the systematic thinking of human experts.

Imagine you are a data analyst. Your task is to select a total of $< n >$ fake users from various attack types to **maximize the overall hit rate**.

Goal-focused Prompt. Emphasizes the core objective of maximizing hit rate while downplaying other distracting conditions. Through goal reinforcement, this prevents LLMs from deviating from the main task due to excessive consideration of secondary factors, enhancing goal-oriented focus.

You will receive information about fake users from various attack types. Your goal is to select a total of $< n >$ users to **maximize the overall hit rate**. Focus on the core objective of maximizing hit rate.

Priority Prompt. Suggests prioritizing attack types with higher hit rates, providing heuristic guidance. This simplifies decision complexity, is suitable for scenarios requiring rapid response, and simultaneously tests LLMs' execution efficiency regarding priority rules.

You need to select a total of $< n >$ fake users from various attack types. Prioritize selecting users from attack types with the **highest hit rates**.

Calculation Prompt. Encourages calculation of cumulative hit rates and promotes numerical reasoning abilities. Forces LLMs to perform numerical reasoning and mathematical modeling (such as sorting, weighted summation), enhances their capability to process quantitative metrics, and ensures that decision results have interpretability and rigor.

Please select a total of $< n >$ fake users from various attack types. For each attack type, calculate the cumulative **hit rate** for all users, then select users to **maximize the total hit rate**.

These prompt designs actually explore different methods for activating LLM capabilities. Each prompt maintains the same core constraints and format requirements, with differences primarily reflected in the methods for guiding thinking and decision-making. Through rigorous experimental evaluation of these prompt variants, we not only optimized the decision-making performance of AgentAttack but also gained valuable insights into the relationship between prompt design and attack performance, providing important references for the application of LLMs in decision tasks.

4.4. Feedback memory module

Effective attack strategies require learning from past experience and continuously improving to address changing defense strategies. Based

on this, we designed a feedback memory module that enables AgentAttack to accumulate experience and improve attack strategies.

Implementing this goal faces technical challenges mainly reflected in three aspects: designing methods for effectively storing multi-dimensional attack data, creating precise evaluation mechanisms for attack effectiveness, and utilizing historical data to guide future decisions. To solve these problems, we designed a multi-level memory architecture that includes three core components:

User Distribution. Responsible for storing user selection distributions for different attack types, capturing strategy combination patterns. After each attack, the system records the specific user IDs that employed various attack methods.

Historical Hit Rates. Stores the actual effects of various attack combinations, providing quantitative performance metrics. Hit rate serves as the primary evaluation metric, directly reflecting the probability of the target item successfully entering the recommendation list.

Effectiveness Classification. Categorizes attack results into different levels, facilitating quick strategy comparison. This classification method simplifies the understanding and comparison process of complex numerical values, enabling the decision system to more intuitively identify patterns of successful and failed strategies.

The feedback memory module records and manages key data from each attack attempt. This architectural design has been optimized to efficiently store and retrieve complex multi-dimensional data.

We also designed a threshold mechanism to objectively evaluate attack effectiveness. This mechanism represents an important improvement over fixed threshold methods, capable of automatically adapting to different attack environments:

$$\text{Hit Rate Category} = \begin{cases} \text{High,} & \text{if hit rate} \geq \tau_{\text{high}} \\ \text{Low,} & \text{if hit rate} \leq \tau_{\text{low}} \\ \text{Medium,} & \text{otherwise,} \end{cases} \quad (8)$$

where τ_{high} and τ_{low} represent dynamic evaluation thresholds, determined directly from the performance range of the current attack pool. This threshold method ensures that evaluation standards remain consistent with the actual performance range of the current attack pool, providing an objective benchmark for effect classification.

Through the feedback memory module, AgentAttack achieves human-like experience accumulation and learning capability, which is an important advantage that enables the system to continuously adapt to changing defense strategies and steadily improve attack effectiveness.

4.5. Composite attack implementation

The planned attack strategies ultimately need to be effectively implemented in the target system. The action module is responsible for seamlessly integrating the fake users selected by the planning module into the target recommendation system. Directly modifying original data or simply appending fake users easily triggers integrity checks or causes ID distribution anomalies, thereby being identified by defense systems. To address this challenge, we designed a matrix operation-based data fusion method that integrates the original interaction matrix M_{original} and the fake user interaction matrix M_{fake} into a poisoned dataset:

$$M_{\text{poisoned}} = \begin{bmatrix} M_{\text{original}} \\ M_{\text{fake}} \end{bmatrix}. \quad (9)$$

We use sparse matrices to represent interaction data, reducing memory usage and making it suitable for large-scale recommendation systems, allowing efficient operation even in resource-limited environments. Vertical stacking preserves original data integrity, avoiding security alerts that might be triggered by direct modifications, enhancing concealment. This method maintains data structure consistency, allowing new users to naturally integrate into the system, effectively avoiding detection mechanisms based on structural anomalies.

In recommender systems, with the rise of data-driven methods, attackers successfully hide counterfeit data by injecting fake data, thereby

Table 2
Statistics of the datasets used in the experiments.

Dataset	#Users	#Items	#Interactions	Density (%)
ML-100k	943	1682	100,000	6.305
ML-1M	6040	3706	1,000,209	4.468
Douban	2848	39,586	894,887	0.794

manipulating recommendation results. Common defense mechanisms typically detect inconsistencies based on data structure, rating distributions, or user behavior anomalies (Wang et al., 2025b). Sparse matrix operations, especially the vertical stacking method, further enhance concealment by introducing new rows without altering the original row content, avoiding detection by these defense mechanisms.

In the vertical stacking method, we assume that the item distribution remains unchanged, meaning that the introduction of fake users does not significantly alter the item distribution in the recommendation system. This assumption simplifies the experimental design, but we recognize that in real-world scenarios, the item distribution may change. In future research, we will explore more complex data injection and fusion strategies.

User interaction extraction is also a key step in implementing attacks. Data from different attack types have varied feature distributions and structural characteristics; simple merging could lead to inconsistencies or anomaly detection. We utilize user IDs returned by the Planning module to precisely extract corresponding data from interaction matrices of various attack models, ensuring structural consistency.

In the final stage of attack implementation, we evaluate effectiveness through multiple metrics, feeding results back to the memory module to form a closed-loop learning system. Data from the implementation module and experience in the memory module create a tight information flow. Attack experience accumulated in memory provides historical references for planning, while planning decisions transform into new actions through implementation. This module linkage simulates the human “attempt-reflection-improvement” learning process, enabling AgentAttack to summarize experience from each attack, identify successful patterns, avoid repeating failures, and achieve continuous adaptation in adversarial environments.

5. Experiments

In this section, we evaluate the effectiveness of AgentAttack. Specifically, we aim to answer the following research questions, following the guidelines emphasized in recent literature on the importance of multi-modal and diverse baselines in evaluating recommender systems’ performance (Dutta et al., 2025): (Q1): How does AgentAttack compare to other attacks in terms of attack effectiveness? (Q2): What impact does the feedback memory module have on improving attack effectiveness? (Q3): How do different LLM models affect the attack performance of AgentAttack? (Q4): How do different prompt engineering techniques influence the generation of attack strategies by LLM agents? (Q5): How do various detection models perform in identifying different attack strategies, especially AgentAttack?

5.1. Experimental setup

Datasets. We evaluate AgentAttack on widely used public recommendation datasets: MovieLens-100K, MovieLens-1M and Douban. Table 2 shows the statistics of these datasets.

Evaluation Metrics. We employ the following attack-oriented metrics:

- **Hit Rate:** Measures the frequency of target items appearing in the recommendation list.
- **Precision:** Quantifies the proportion of successful target item recommendations.

- **Recall:** Evaluates the coverage of target items among all potential recommendations.
- **NDCG:** Assesses the ranking quality of target items in the recommendation list.

Attack Methods. We evaluate the effectiveness of AgentAttack through comparison with various baseline methods, including traditional fixed-pattern attacks (Random Attack, Average Attack, Bandwagon Attack, Probe Attack), end-to-end learning methods (AUSH, GOAT, CLeaR), and hybrid attack methods (Hybrid Attack).

- **Random Attack** (Lam & Riedl, 2004): A simple method that randomly selects filler items and interacts with target items to generate fake user profiles.
- **Average Attack** (Burke et al., 2006): Simulating average interaction patterns to create more realistic fake user profiles, improving attack effectiveness.
- **Bandwagon Attack** (O’Mahony et al., 2005): Enhances similarity by leveraging the popularity of trending items, making fake profiles appear more credible through interactions with popular items.
- **Probe Attack** (Hurley et al., 2007): Builds profiles highly similar to real users by systematically analyzing system responses.
- **AUSH** (Lin et al., 2020): Uses generative adversarial networks to create fake user profiles that effectively promote target items while evading detection.
- **GOAT** (Wu et al., 2021a): A gradient-based attack method specifically designed for graph neural network recommender systems, optimizing malicious interactions through gradient information.
- **Hybrid Attack** (Zhang, 2011): A composite strategy that combines multiple attack patterns to leverage the complementary strengths of different attack methods.
- **CLeaR** (Wang et al., 2024d): Exploits contrastive learning vulnerabilities by promoting smoother spectral distributions and optimizing target item rankings through bi-level optimization.

Recommendation Systems. We evaluate attacks against different recommendation architectures:

- **LightGCN** (He et al., 2020): A graph-based collaborative filtering model.
- **GMF** (Koren et al., 2009): Generalized Matrix Factorization method.
- **XSimGCL** (Yu et al., 2023): Recommendation model based on contrastive learning.

Implementation Details. We implement the attack and recommendation models using PyTorch. Unless otherwise specified, all methods follow the recommended parameter settings. Specifically, we set the following experimental configurations:

- **Model Initialization:** We initialize the model parameters using the Xavier initialization method. The embedding dimension for all models is set to 64.
- **Training Settings:** The batch size for all models is set to 2048 to balance memory constraints and training stability. We use the Adam optimizer with a learning rate of 0.005. The training is set to a maximum of 30 epochs to ensure adequate convergence while avoiding overfitting. For LightGCN, we set the number of GNN layers to 2; for XSimGCL, we use a 2-layer encoder with a contrastive learning loss weight of 0.05, a temperature parameter of 0.2, and perturbation noise of 0.1.
- **Variance Control and Regularization:** We use L2 regularization to control the complexity of the model, ensuring the embeddings do not overfit the training data, with a regularization weight set to $1e-4$. Additionally, we set a dropout rate of 0.3 to further prevent overfitting.
- **Evaluation:** We evaluate the top 50 recommendations for each user. During the training of the recommendation models, the performance of each model is tracked over multiple training epochs, and the best-performing model is saved.

Table 3
Top-50 performance comparison.

Base Model	Method	ML-100k				ML-1M				Douban			
		HR (%)	Prec (%)	Rec (%)	NDCG (%)	HR (%)	Prec (%)	Rec (%)	NDCG (%)	HR (%)	Prec (%)	Rec (%)	NDCG (%)
GMF	ProbeAttack	0.336	0.037	0.378	0.139	0.217	0.097	0.976	0.609	0.146	0.021	0.218	0.078
	RandomAttack	0.227	0.022	0.227	0.086	0.225	0.093	0.935	0.588	0.141	0.031	0.313	0.127
	AverageAttack	0.222	0.022	0.222	0.082	0.236	0.096	0.962	0.759	0.200	0.045	0.457	0.241
	BandwagonAttack	0.222	0.022	0.222	0.082	0.213	0.091	0.905	0.715	0.204	0.066	0.668	0.380
	AUSH	0.277	0.027	0.277	0.103	0.209	0.099	0.999	0.992	0.203	0.099	0.993	0.989
	GOAT	0.231	0.023	0.231	0.089	0.216	0.098	0.984	0.738	0.211	0.076	0.768	0.591
	HybridAttack	0.218	0.021	0.218	0.081	0.234	0.097	0.976	0.708	0.141	0.031	0.308	0.154
	CLear	0.269	0.026	0.269	0.106	0.229	0.081	0.816	0.473	0.182	0.035	0.355	0.160
LightGCN	AgentAttack	0.344	0.034	0.344	0.140	0.243	0.100	1.002	0.752	0.190	0.062	0.627	0.427
	ProbeAttack	0.134	0.013	0.134	0.044	0.201	0.098	0.988	0.957	0.163	0.037	0.378	0.150
	RandomAttack	0.126	0.013	0.130	0.046	0.208	0.102	1.015	0.922	0.121	0.037	0.376	0.176
	AverageAttack	0.105	0.011	0.105	0.036	0.208	0.103	1.028	1.010	0.036	0.009	0.098	0.056
	BandwagonAttack	0.076	0.007	0.076	0.026	0.218	0.104	1.036	1.013	0.208	0.102	1.028	1.012
	AUSH	0.135	0.013	0.134	0.046	0.211	0.101	1.012	1.005	0.228	0.107	1.077	1.042
	GOAT	0.084	0.008	0.084	0.028	0.211	0.100	1.001	0.942	0.191	0.068	0.688	0.388
	HybridAttack	0.147	0.014	0.147	0.051	0.213	0.102	1.025	0.971	0.162	0.068	0.687	0.493
XSimGCL	CLear	0.096	0.009	0.096	0.032	0.194	0.068	0.682	0.419	0.205	0.031	0.310	0.152
	AgentAttack	0.193	0.019	0.193	0.078	0.223	0.104	1.040	0.967	0.221	0.094	0.941	0.844
	ProbeAttack	1.952	0.199	1.993	0.792	0.202	0.099	0.995	0.973	0.104	0.030	0.301	0.111
	RandomAttack	2.039	0.207	2.073	0.823	0.356	0.171	1.706	1.372	0.581	0.274	2.742	1.803
	AverageAttack	1.939	0.197	1.976	0.784	0.026	0.007	0.070	0.020	0.148	0.061	0.617	0.275
	BandwagonAttack	2.002	0.202	2.023	0.807	0.316	0.145	1.450	1.101	0.589	0.253	2.531	1.545
	AUSH	1.939	0.196	1.964	0.779	0.352	0.164	1.640	1.302	0.753	0.345	3.459	2.128
	GOAT	1.640	0.168	1.682	0.654	0.134	0.060	0.600	0.416	0.148	0.064	0.647	0.273
XSimGCL	HybridAttack	2.136	0.217	2.174	0.851	0.223	0.099	0.995	0.673	0.269	0.116	1.166	0.573
	CLear	2.128	0.214	2.145	0.867	0.209	0.099	0.997	0.478	0.156	0.052	0.529	0.198
	AgentAttack	2.250	0.231	2.313	0.898	0.362	0.172	1.725	1.269	0.914	0.403	4.032	2.308

5.2. Comparison of attack effectiveness (Q1)

We compare AgentAttack with individual attack methods across different recommendation systems. Additionally, we evaluate our approach against CLear (Wang et al., 2024d), a recent end-to-end method, and a hybrid attack strategy that randomly integrates Bandwagon Attack, Random Attack, and Average Attack. Table 3 shows the hit rate@50, precision@50, recall@50, and NDCG@50 values achieved by each attack method on different datasets targeting various recommendation systems. For each attack method, we conduct five runs and calculate the average of the metrics.

By comparing the performance of different attack methods across three datasets and three recommendation models, we can clearly observe that:

- AgentAttack demonstrates superiority in most test scenarios. Out of 9 test scenarios, AgentAttack achieved the best attack effectiveness in 6 scenarios, particularly excelling in all XSimGCL model tests. Even in scenarios where it did not achieve the best results, AgentAttack maintained performance close to the optimal methods, indicating its high stability and versatility. This is primarily attributed to its learning mechanism, which can adjust attack strategies according to the characteristics of different recommendation systems.
- Traditional fixed-pattern attack methods such as Probe Attack perform adequately in certain scenarios but generally lack flexibility and cannot effectively address different data features and model structures. Learning-based attack methods like AUSH and GOAT, while more effective than simple attack techniques, still show gaps compared to AgentAttack, especially with more complex model structures.
- Hybrid Attack outperforms individual attack methods in most cases, but AgentAttack consistently surpasses Hybrid Attack's performance. This finding demonstrates that AgentAttack, with its broader attack strategy repository, is more effective than simple random mixing

strategies by intelligently routing to the more effective attack patterns.

- Cross-dataset analysis shows significant performance variations of the same attack methods across different datasets, highlighting the strong dependency of attack methods on data characteristics. AgentAttack effectively mitigates this limitation through its adaptive framework.

These results confirm the necessity of designing adaptive attack frameworks and demonstrate the effectiveness of AgentAttack in addressing diverse recommendation systems.

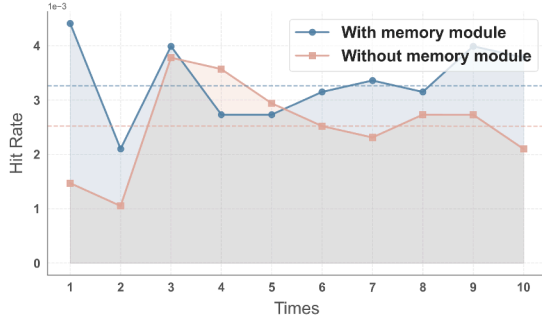
5.3. Effectiveness of feedback memory module (Q2)

We choose Qwen-plus¹ and observe the impact of the feedback memory module on attacks using the ML-100k dataset. Fig. 3 shows the evolution of attack effectiveness over 10 iterations with and without the feedback memory module across different recommendation models.

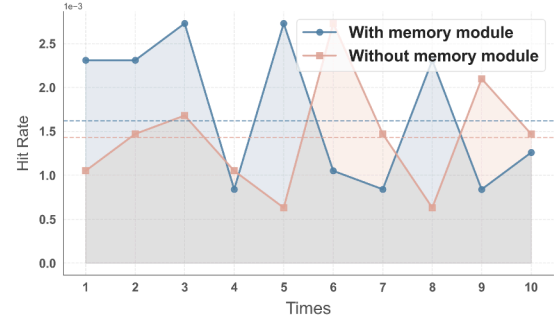
In Fig. 3, dashed lines represent the average hit rate@50. Based on Fig. 3, we can derive the following analysis:

- The results indicate that, for all three recommendation models, introducing the feedback memory module yields higher average hit rates compared to attacks without the memory module. This confirms that the memory mechanism enhances the hit rates of attacks by accumulating and using historical interaction information to optimize attack strategies.
- Additionally, the feedback memory module reduces the occurrence of extremely low values to some extent. Due to the instability of LLMs and differences in recommendation models during each training run, the hit rate curves exhibit certain fluctuations. The curves with the feedback memory module show less volatility than those without it,

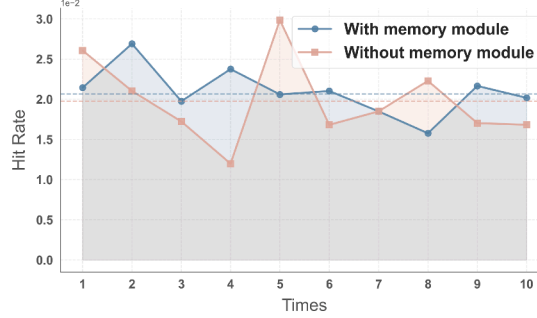
¹ <https://tongyi.aliyun.com/qianwen/>



(a) Comparison of GMF



(b) Comparison of LightGCN



(c) Comparison of XSimGCL

Fig. 3. Hit rate@50 comparison with and without memory module.

demonstrating that the introduction of the feedback memory module enables AgentAttack to maintain more consistent attack performance, reduce random influences, and enhance attack reliability.

- Analyzing the curve trends reveals that in some recommendation models like XSimGCL, attacks with the memory module display more stable trends in later iterations, while versions without the memory module show greater fluctuations. This indicates that the memory module can maintain and accumulate advantages during long-term iterations, which is significant for understanding the persistence of attacks.

Fig. 4 shows the results of 40 experiments, which only include the attack curve generated using the feedback memory module. Fig. 4a demonstrates the change in raw hit rate as the number of iterations increases. This chart reflects the variability of the data without any processing. We can observe that in the early stages of the experiment, the hit rate fluctuates significantly, showing high instability, particularly in the first few experiments. Fig. 4b presents the change in hit rate after applying a rolling average to every five experiments. This smoothing process makes the chart more stable, revealing the long-term trend and more clearly showing the impact of the feedback memory module.

It can be seen that in the initial stages of the experiment, the hit rate exhibits high variability, especially between the 5th and 10th experiments. As the experiments progress, extremely low values gradually decrease. This indicates that the memory module, by accumulating and utilizing historical interaction information, progressively optimizes the attack strategy, reducing extreme fluctuations caused by model instability or random factors. At the same time, we can observe that the hit rate shows an overall upward trend, indicating that the feedback memory module plays a cumulative role over long-term operation. With more operations, the memory module helps the system gradually enhance stability, reduce fluctuations, and improve the effectiveness of the attack strategy.

Table 4

Top-50 performance comparison of different LLMs.

Dataset	LLM Model	HR (%)	Prec (%)	Rec (%)	NDCG (%)
ML-100k	Qwen-plus	0.193	0.019	0.193	0.078
	DeepSeek-r1	0.159	0.016	0.159	0.060
	DeepSeek-v3	0.100	0.010	0.100	0.034
	Doubao-1.5pro	0.168	0.016	0.168	0.067
ML-1M	Qwen-plus	0.223	0.104	1.040	0.967
	DeepSeek-r1	0.214	0.102	1.029	1.013
	DeepSeek-v3	0.219	0.104	1.040	1.004
	Doubao-1.5pro	0.213	0.102	1.020	0.888
Douban	Qwen-plus	0.221	0.094	0.941	0.844
	DeepSeek-r1	0.221	0.100	1.008	0.907
	DeepSeek-v3	0.217	0.093	0.934	0.832
	Doubao-1.5pro	0.220	0.102	1.024	0.943

5.4. Comparison of different LLMs (Q3)

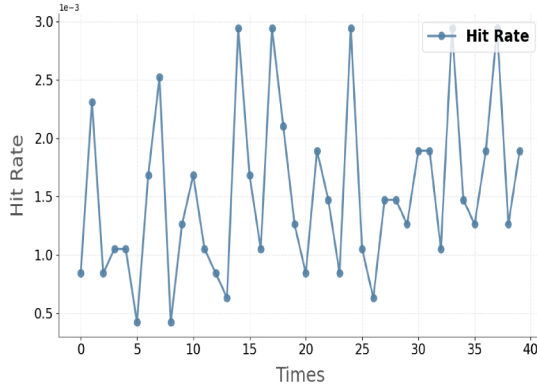
In addition to Qwen-plus, we test other LLMs (including DeepSeek-r1², DeepSeek-v3³, and Doubao-1.5pro⁴) as the core reasoning backbone for the AgentAttack planning module. As shown in Table 4, there are certain performance differences among various LLMs across different datasets. Based on these differences, we can draw the following conclusions:

- From the table, DeepSeek-v3 with fewer parameters generally performs worse than models with larger parameter scales, which indicates that model capacity has a significant impact on performance

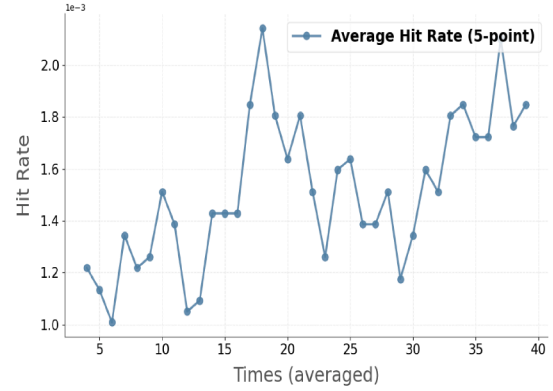
² <https://www.deepseek.com/>

³ <https://www.deepseek.com/>

⁴ <https://www.volcengine.com/product/doubao>



(a) Variation in Hit Rate



(b) Variation in Average Hit Rate

Fig. 4. Comparison of hit rate@50 variations over 40 iterations.

in recommendation system tasks. Using LLMs with larger parameter counts and better reasoning capabilities can achieve more effective attack results.

- Qwen-plus performs best on the ML-100k dataset, while DeepSeek-r1 achieves the highest NDCG metric on the ML-1M dataset, and Doubao-1.5pro shows slight advantages in Recall and Precision on the Doubao dataset.

These results suggest that once LLMs reach a certain scale and capability level, their performance differences as reasoning backbones depend on their grasp of domain-specific knowledge and adaptability to reasoning patterns. Therefore, in practical applications, one can select appropriate LLMs based on specific task characteristics and resource constraints, while further optimizing model performance through targeted prompt engineering and contextual learning.

5.5. The impact of prompt engineering (Q4)

Prompt engineering plays a crucial role in optimizing the performance of LLMs. The design of prompts can significantly impact the model's task execution ability and response quality, especially in complex task environments (Sahoo et al., 2024). Prompt strategies are not merely simple rewrites of task descriptions; they involve how to establish effective interactions between the model and the task, which is essential for enhancing the model's performance. Specific prompt designs can guide the model's focus on key task elements, stimulating more effective reasoning and decision-making, leading to more efficient task processing.

In this section, we focus on how different prompt strategies affect the performance of AgentAttack and hypothesize that these strategies have a complex interaction with the model's cognitive preferences, task requirements, and architecture adaptability. Through experiments, we validate this hypothesis and find that different prompt strategies exhibit significant adaptability differences across models, further confirming that prompt engineering should not adopt a "one-size-fits-all" approach but should be optimized according to the characteristics of the model (Wang et al., 2024a).

To evaluate how different prompt designs impact the performance of AgentAttack, we conduct experiments using ten prompt variants. Additionally, we compare ReAct Prompting (Yao et al., 2023) and Reflexion Prompting (Shinn et al., 2023) as baseline strategies. For each prompt, we perform 10 experiments and calculate the average results. Table 5 presents the evaluation metrics for each type of prompt.

In Table 5, we find that Qwen-plus exhibits clear preferences in its responses to different prompt strategies. Goal-focused Prompt achieves the best performance across all evaluation metrics. Step-by-step Prompt and

Table 5

Top-50 performance of Qwen-plus with different prompt strategies.

Prompt Strategy	HR (%)	Prec (%)	Rec (%)	NDCG (%)
Basic Prompt	0.128	0.013	0.128	0.047
Comparative Prompt	0.160	0.016	0.164	0.062
Analytical Prompt	0.137	0.014	0.137	0.052
Step-by-step Prompt	0.173	0.019	0.185	0.071
Exploratory Prompt	0.133	0.014	0.143	0.053
Minimalist Prompt	0.147	0.015	0.154	0.059
Expert Role Prompt	0.154	0.015	0.154	0.058
Goal-focused Prompt	0.179	0.018	0.183	0.070
Priority Prompt	0.147	0.015	0.154	0.058
Calculation Prompt	0.107	0.013	0.126	0.052
ReAct Prompting	0.170	0.017	0.170	0.065
Reflexion Prompting	0.122	0.012	0.122	0.045

Table 6

Top-50 performance of Doubao-1.5pro model with different prompt strategies.

Prompt Strategy	HR (%)	Prec (%)	Rec (%)	NDCG (%)
Basic Prompt	0.151	0.015	0.154	0.060
Comparative Prompt	0.135	0.014	0.135	0.050
Analytical Prompt	0.166	0.017	0.173	0.067
Step-by-step Prompt	0.130	0.013	0.130	0.045
Exploratory Prompt	0.242	0.025	0.248	0.101
Minimalist Prompt	0.114	0.012	0.116	0.042
Expert Role Prompt	0.173	0.017	0.173	0.066
Goal-focused Prompt	0.145	0.015	0.147	0.056
Priority Prompt	0.175	0.018	0.175	0.072
Calculation Prompt	0.187	0.019	0.194	0.073
ReAct Prompting	0.202	0.021	0.210	0.076
Reflexion Prompting	0.130	0.014	0.135	0.046

Comparative Prompt also demonstrate good performance, while Calculation Prompt performs worst, ranking lowest in all metrics.

To further investigate the differences in how different model architectures respond to prompt strategies, we use Doubao-1.5pro as the backbone, with results shown in Table 6. Doubao-1.5pro demonstrates exceptional adaptability to Exploratory Prompt, significantly outperforming all other strategies across all metrics. In contrast, this model shows poor performance with Minimalist Prompt and Step-by-step Prompt.

Through comprehensive comparison, we discover several interesting patterns:

- **Model-specific Strategy Adaptability.** The same prompt strategy exhibits significantly different performance across different large language model architectures, indicating that prompt engineering

Table 7
Detection performance comparison.

Detect Model	Method	ML-100k				ML-1M				Douban			
		Accuracy	Recall	Precision	F1	Accuracy	Recall	Precision	F1	Accuracy	Recall	Precision	F1
BaseDetect	BandwagonAttack	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.991	0.600	0.500	0.545
	AUSH	1.000	1.000	1.000	1.000	0.998	0.833	1.000	0.909	0.995	0.600	0.750	0.667
	HybridAttack	1.000	1.000	1.000	1.000	0.998	0.833	1.000	0.909	0.988	1.000	0.417	0.588
	AgentAttack	0.984	1.000	1.000	1.000	0.998	0.917	0.846	0.878	0.988	0.800	0.400	0.533
DHAGCN	BandwagonAttack	1.000	1.000	1.000	1.000	0.679	1.000	0.030	0.058	0.991	0.200	0.500	0.286
	AUSH	1.000	1.000	1.000	1.000	0.693	0.917	0.029	0.056	0.991	0.400	0.500	0.444
	HybridAttack	1.000	1.000	1.000	1.000	0.706	0.917	0.030	0.058	0.995	0.800	0.667	0.727
	AgentAttack	0.990	1.000	0.500	0.670	0.696	0.917	0.029	0.056	0.988	0.200	0.250	0.222
PCASelectUsers	BandwagonAttack	0.661	1.000	0.015	0.030	0.631	1.000	0.026	0.051	0.590	0.800	0.017	0.033
	AUSH	0.667	1.000	0.016	0.031	0.573	0.083	0.002	0.004	0.587	0.600	0.013	0.025
	HybridAttack	0.561	1.000	0.012	0.024	0.597	0.417	0.010	0.020	0.621	0.600	0.014	0.027
	AgentAttack	0.622	1.000	0.014	0.027	0.589	0.417	0.010	0.020	0.612	0.200	0.005	0.009

cannot adopt a “one-size-fits-all” approach, but requires optimization for specific models.

- **Model Differentiation in Cognitive Styles.** Different models demonstrate distinct “cognitive preferences”—some models are better adapted to structured, procedural prompt styles, while others perform better under open-ended, exploratory prompts. This provides important guidance for prompt strategy selection in AgentAttack.
- **Performance and Comparison of Baseline Strategies.** Both ReAct Prompting and Reflexion Prompting demonstrate competitive performance, but they still do not surpass the best-performing experimental prompts. In both models, ReAct Prompting outperforms Reflexion Prompting, indicating that prompt strategies with clear interactive feedback (i.e., ReAct Prompting) align better with the cognitive styles of both models. Although Reflexion Prompting is widely used in the literature, it does not significantly surpass other strategies in this experiment.
- **Robustness Differences in Prompt Strategies.** Some prompt strategies exhibit strong cross-model generalizability, while others are highly dependent on specific model characteristics. This finding has significant implications for improving the robustness of AgentAttack, suggesting that strategies with better cross-model stability should be prioritized.

The experimental results demonstrate that prompt strategy selection has a significant impact on the performance of AgentAttack. The dramatically different capabilities shown by different models in handling specific types of tasks reveal specialization phenomena in certain cognitive abilities. This emphasizes that in Agent system design, prompt strategies must be optimized according to specific model characteristics to achieve optimal performance.

5.6. Comparison of detection performance (Q5)

In this section, we evaluate the detection effectiveness of different attack methods using three detection models: BaseDetect (Popescu et al., 2009), DHAGCN (Hao et al., 2023), and PCASelectUsers (Mehta & Nejd, 2009). Our primary goal is to analyze the performance of these models in detecting various attack strategies, particularly comparing the detection effectiveness of AgentAttack with other attack methods.

BaseDetect (Popescu et al., 2009) is a neural network-based detection method that classifies based on learning the differences between normal user behavior and attack behavior. It uses a simple binary classification neural network, balancing the data using oversampling or undersampling strategies, and optimizing the detection threshold based on the F1 score. DHAGCN (Hao et al., 2023) is a graph convolutional network (GCN) method that builds a similarity graph between users and uses graph convolution to capture relationships within the graph structure to detect anomalous user behavior. PCASelectUsers (Mehta & Nejd, 2009) is based on principal component analysis (PCA) for dimensional-

ity reduction, selecting user features strongly correlated with attack patterns for detection. These three methods, with their different principles and characteristics, provide a comprehensive evaluation of detection performance.

According to the experimental results shown in Table 7:

- AgentAttack consistently outperforms traditional fixed-pattern attacks, such as BandwagonAttack and HybridAttack, in most scenarios. Its ability to adjust its attack strategy based on the environment allows it to evade detection more effectively, as evidenced by the lower detection recall and precision values for AgentAttack compared to other attacks.
- DHAGCN, as a graph-based detection model, performs well with attacks like BandwagonAttack but struggles with the more adaptive AgentAttack. This highlights the limitations of graph-based methods in detecting dynamic and evolving attack strategies.
- PCASelectUsers, which relies on dimensionality reduction and interaction strength, encounters significant difficulties in detecting malicious users, underperforming compared to both BaseDetect and DHAGCN. This indicates that static models, like PCA, are not well-suited for detecting complex attack behaviors.
- Traditional hybrid and learning-based attacks, such as HybridAttack and AUSH, show good detection results across most models, but AgentAttack surpasses these methods by intelligently selecting attack strategies based on real-time feedback.

In conclusion, AgentAttack demonstrates superior evasiveness compared to traditional and hybrid attack methods. While simpler models like BaseDetect are effective in detecting fixed-pattern attacks, more complex attacks like AgentAttack highlight the limitations of current detection systems. The results emphasize the need for more sophisticated detection strategies that can handle evolving attack behaviors effectively.

6. Conclusion

In this paper, we investigate the potential of LLM-based agents in shilling attacks against recommender systems. The proposed AgentAttack framework generates and optimizes attack strategies through the coordinated operation of four key modules: attack profile, planning, feedback memory, and action. Extensive experiments across multiple datasets demonstrate that AgentAttack exhibits notable advantages over traditional and end-to-end learning-based attack methods, with both prompt engineering techniques and different LLM backbones influencing attack effectiveness.

Although AgentAttack demonstrates the powerful capabilities of intelligent agents in adversarial settings, potential defense strategies and their limitations must also be considered. Some defense measures, such

as model regularization, anomaly detection, and ensemble methods, can effectively detect and mitigate the impact of dynamic attacks. However, these defense mechanisms often face challenges when dealing with highly flexible and context-sensitive attacks, as these attacks can evolve in real time, making it difficult for defense systems to recognize and respond promptly. Therefore, a dual focus on both offense and defense is crucial for the long-term security of recommender systems. A comprehensive offensive-defense perspective will ensure that more holistic strategies are adopted in safeguarding the security of recommendation systems (Alnabulsi et al., 2024).

Building on these findings, we argue that the application of intelligent agents in security domains presents both new challenges and opportunities for recommender system defense mechanisms. For future work, we plan to: (1) introduce an objective function or fitness measure to strengthen attack designs and achieve a more rigorous evaluation and optimization of the user selection mechanism, (2) investigate more efficient memory mechanisms to enhance long-term attack strategy optimization, (3) further improve the stability and effectiveness of LLM agent-based attacks, and (4) explore proactive defense measures against agent-driven attacks while extending the framework to emerging recommendation paradigms.

CRedit authorship contribution statement

Chenyu Li: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing – original draft, Visualization; **Zongwei Wang:** Methodology, Software, Validation, Writing – review & editing; **Hao Ma:** Software, Data curation, Investigation, Writing – review & editing; **Wei Li:** Resources, Validation, Writing – review & editing; **Huan Wu:** Resources, Validation, Writing – review & editing; **Yuqi Song:** Methodology, Formal analysis, Writing – review & editing; **Min Gao:** Conceptualization, Supervision, Project administration, Funding acquisition, Writing – review & editing.

Data availability

The datasets supporting the conclusions of this article are publicly available. MovieLens datasets (100K and 1M) can be downloaded from GroupLens Research (<https://grouplens.org/datasets/movielens/>). The Douban dataset was obtained from Zhao et al. (2016).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the National Key Research and Development Program of China 2024YFC3014900, the National Natural Science Foundation of China 62176028, the Major Scientific and Technological Project of Chongqing City Vocational College (XJKJ202300002) and the Program of China Scholarship Council (202406050164).

References

Aggarwal, C. C. et al. (2016). Recommender systems (vol. 1). Springer.
 Ali, A., & Ghanem, M. C. (2025). Beyond detection: Large language models and next-generation cybersecurity. *SHIFRA*, 2025, 81–97.
 Alnabulsi, H., Islam, R., Alsmadi, I., & Bevinakoppa, S. (2024). An innovative method of malicious code injection attacks on websites. *Applied Data Science and Analysis*, 2024, 39–51.
 Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A. et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
 Burke, R., Mobasher, B., Williams, C., & Bhaumik, R. (2006). Detecting profile injection attacks in collaborative recommender systems. In *The 8th IEEE international conference on e-commerce technology and the 3rd IEEE international conference on enterprise computing, e-commerce, and e-services (CEC/EEE'06)* (pp. 23). IEEE.

Cheng, Y., Zhang, C., Zhang, Z., Meng, X., Hong, S., Li, W., Wang, Z., Wang, Z., Yin, F., Zhao, J. et al. (2024). Exploring large language model based intelligent agents: Definitions, methods, and prospects. *arXiv preprint arXiv:2401.03428*.
 Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S. et al. (2023). Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24 (240), 1–113.
 Deldjoo, Y., Nazary, F., Ramisa, A., Mcauley, J., Pellegrini, G., Bellogin, A., & Noia, T. D. (2023). A review of modern fashion recommender systems. *ACM Computing Surveys*, 56(4), 1–37.
 Dutta, P. K., Kumar, A., Sakici, S., & Mensah, B. (2025). Enhancing point-of-interest recommendation systems through multi-modal data integration in location-based social networks: Challenges and future directions. *EDRAAK*, 2025, 12–18.
 Gao, C., Wang, X., He, X., & Li, Y. (2022). Graph neural networks for recommender system. In *Proceedings of the fifteenth ACM international conference on web search and data mining* (pp. 1623–1625).
 Ge, Y., Liu, S., Fu, Z., Tan, J., Li, Z., Xu, S., Li, Y., Xian, Y., & Zhang, Y. (2024). A survey on trustworthy recommender systems. *ACM Transactions on Recommender Systems*, 3(2), 1–68.
 Gunes, I., Kaleli, C., Bilge, A., & Polat, H. (2014). Shilling attacks against recommender systems: A comprehensive survey. *Artificial Intelligence Review*, 42, 767–799.
 Hao, Y., Meng, G., Wang, J., & Zong, C. (2023). A detection method for hybrid attacks in recommender systems. *Information Systems*, 114, 102154.
 He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., & Wang, M. (2020). LightGCN: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval* (pp. 639–648).
 He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T.-S. (2017). Neural collaborative filtering. In *Proceedings of the 26th international conference on world wide web* (pp. 173–182).
 He, X., Liu, S., Keung, J., & He, J. (2024). Co-clustering for federated recommender system. In *Proceedings of the ACM web conference 2024* (pp. 3821–3832).
 Hu, Y., Koren, Y., & Volinsky, C. (2008). Collaborative filtering for implicit feedback datasets. In *2008 Eighth IEEE international conference on data mining* (pp. 263–272). IEEE.
 Huang, X., Lian, J., Lei, Y., Yao, J., Lian, D., & Xie, X. (2023). Recommender AI agent: Integrating large language models for interactive recommendations. *arXiv preprint arXiv:2308.16505*.
 Hurley, N. J., O'Mahony, M. P., & Silvestre, G. C. M. (2007). Attacking recommender systems: A cost-benefit analysis. *IEEE Intelligent Systems*, 22(3), 64–68.
 Hussain, I. (2024). Fortifying AI against cyber threats advancing resilient systems to combat adversarial attacks. *EDRAAK*, 2024, 26–31.
 Jesse, M., & Jannach, D. (2021). Digital nudging with recommender systems: Survey and future directions. *Computers in Human Behavior Reports*, 3, 100052.
 Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8), 30–37.
 Lam, S. K., & Riedl, J. (2004). Shilling recommender systems for fun and profit. In *Proceedings of the 13th international conference on world wide web* (pp. 393–402).
 Lin, C., Chen, S., Li, H., Xiao, Y., Li, L., & Yang, Q. (2020). Attacking recommender systems with augmented user profiles. In *Proceedings of the 29th ACM international conference on information & knowledge management* (pp. 855–864).
 Lin, C., Chen, S., Zeng, M., Zhang, S., Gao, M., & Li, H. (2022). Shilling black-box recommender systems by learning to generate fake user profiles. *IEEE Transactions on Neural Networks and Learning Systems*, 35(1), 1305–1319.
 Lu, J., Wu, D., Mao, M., Wang, W., & Zhang, G. (2015). Recommender system application developments: A survey. *Decision Support Systems*, 74, 12–32.
 Ma, H., Gao, M., Wei, F., Wang, Z., Jiang, F., Zhao, Z., & Yang, Z. (2024). Stealthy attack on graph recommendation system. *Expert Systems with Applications*, 255, 124476.
 Mehta, B., & Nejdl, W. (2009). Unsupervised strategies for shilling detection and robust collaborative filtering. *User Modeling and User-Adapted Interaction*, 19(1), 65–97.
 Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., & Mian, A. (2023). A comprehensive overview of large language models. *arXiv preprint arXiv:2307.06435*.
 O'Mahony, M. P., Hurley, N. J., & Silvestre, G. C. M. (2005). Recommender systems: Attack types and strategies. In *AAAI* (pp. 334–339).
 Pan, W., & Chen, L. (2013). GBPR: Group preference based bayesian personalized ranking for one-class collaborative filtering. In *IJCAI* (pp. 2691–2697). (vol. 13).
 Park, D. H., Kim, H. K., Choi, I. Y., & Kim, J. K. (2012). A literature review and classification of recommender systems research. *Expert Systems with Applications*, 39(11), 10059–10072.
 Popescu, M.-C., Balas, V. E., Perescu-Popescu, L., & Mastorakis, N. (2009). Multilayer perceptron and neural networks. *WSEAS Transactions on Circuits and Systems*, 8(7), 579–588.
 Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140), 1–67.
 Rosli, M. M. (2025). Privacy-preserving transfer learning for community detection in multiple networks: A review. *Babylonian Journal of Machine Learning*, 2025, 76–85.
 Roy, D., & Dutta, M. (2022). A systematic review and research perspective on recommender systems. *Journal of Big Data*, 9(1), 59.
 Sahoo, P., Singh, A. K., Saha, S., Jain, V., Mondal, S., & Chadha, A. (2024). A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:2402.07927*.
 Schafer, J. B., Frankowski, D., Herlocker, J., & Shilad, S. (2007). Collaborative filtering recommender systems. In *The adaptive web: Methods and strategies of web personalization* (pp. 291–324). Springer.

- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 8634–8652.
- Si, M., & Li, Q. (2020). Shilling attacks against collaborative recommender systems: A review. *Artificial Intelligence Review*, 53, 291–319.
- Sundar, A. P., Li, F., Zou, X., Gao, T., & Russomanno, E. D. (2020). Understanding shilling attacks and their detection traits: A comprehensive survey. *IEEE Access*, 8, 171703–171715.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F. et al. (2023). Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.
- Wang, L., Chen, X., Deng, X., Wen, H., You, M., Liu, W., Li, Q., & Li, J. (2024a). Prompt engineering in consistency and reliability with the evidence-based guideline for LLMs. *NPJ Digital Medicine*, 7(1), 41.
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y. et al. (2024b). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 186345.
- Wang, W., Wang, C., Feng, F., Shi, W., Ding, D., & Chua, T.-S. (2024c). Uplift modeling for target user attacks on recommender systems. In *Proceedings of the ACM web conference 2024* (pp. 3343–3354).
- Wang, X., He, X., Wang, M., Feng, F., & Chua, T.-S. (2019). Neural graph collaborative filtering. In *Proceedings of the 42nd international ACM SIGIR conference on research and development in information retrieval* (pp. 165–174).
- Wang, Z., Gao, M., Li, W., Yu, J., Guo, L., & Yin, H. (2023). Efficient bi-level optimization for recommendation denoising. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 2502–2511).
- Wang, Z., Gao, M., Yu, J., Gao, X., Nguyen, Q. V. H., Sadiq, S., & Yin, H. (2025a). Id-free not risk-free: LLM-powered agents unveil risks in id-free recommender systems. In *Proceedings of the 48th international ACM SIGIR conference on research and development in information retrieval* (pp. 1902–1911).
- Wang, Z., Yu, J., Chen, T., Yin, H., Sadiq, S., & Gao, M. (2025b). Towards secure and robust recommender systems: A data-centric perspective. In *Proceedings of the eighteenth ACM international conference on web search and data mining* (pp. 1020–1023).
- Wang, Z., Yu, J., Gao, M., Yin, H., Cui, B., & Sadiq, S. (2024d). Unveiling vulnerabilities of contrastive recommender systems to poisoning attacks. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 3311–3322).
- Wang, Z., Yu, J., Gao, M., Yuan, W., Ye, G., Sadiq, S., & Yin, H. (2024e). Poisoning attacks and defenses in recommender systems: A survey. *arXiv preprint arXiv:2406.01022*.
- Wu, F., Gao, M., Yu, J., Wang, Z., Liu, K., & Wang, X. (2021a). Ready for emerging threats to recommender systems? A graph convolution-based generative shilling attack. *Information Sciences*, 578, 683–701.
- Wu, J., Wang, X., Feng, F., He, X., Chen, L., Lian, J., & Xie, X. (2021b). Self-supervised graph learning for recommendation. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval* (pp. 726–735).
- Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E. et al. (2025). The rise and potential of large language model based agents: A survey. *Science China Information Sciences*, 68(2), 121101.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). React: Synergizing reasoning and acting in language models. In *International conference on learning representations (ICLR)*.
- Yu, J., Gao, M., Rong, W., Li, W., Xiong, Q., & Wen, J. (2017). Hybrid attacks on model-based social recommender systems. *Physica A: Statistical Mechanics and its Applications*, 483, 171–181.
- Yu, J., Xia, X., Chen, T., Cui, L., Hung, N. Q. V., & Yin, H. (2023). XSIMGCL: Towards extremely simple graph contrastive learning for recommendation. *IEEE Transactions on Knowledge and Data Engineering*, 36(2), 913–926.
- Zaharaddeen, A. H., & Mukhtar, N. (2024). An author-centric framework error minimization in scholarly recommender system (acfemsr). *Babylonian Journal of Internet of Things*, 2024, 161–168.
- Zhang, A., Chen, Y., Sheng, L., Wang, X., & Chua, T.-S. (2024). On generative agents in recommendation. In *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval* (pp. 1807–1817).
- Zhang, F. (2011). Analysis of bandwagon and average hybrid attack model against trust-based recommender systems. In *2011 Fifth international conference on management of e-commerce and e-government* (pp. 269–273). IEEE.
- Zhang, S., Yao, L., Sun, A., & Tay, Y. (2019). Deep learning based recommender system: A survey and new perspectives. *ACM Computing Surveys*, 52(1), 1–38.
- Zhao, A., Huang, D., Xu, Q., Lin, M., Liu, Y.-J., & Huang, G. (2024). Expel: LLM agents are experiential learners. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 19632–19642). (vol. 38).
- Zhao, G., Qian, X., & Xie, X. (2016). User-service rating prediction by exploring social users' rating behaviors. *IEEE Transactions on Multimedia*, 18(3), 496–506.